

MongoDB Aggregation Pipelines

Each question is shown in the blue box, followed by its answer. Commands are for mongosh. Every answer includes small sample data so you can test the pipeline yourself. Outputs are examples, and dates assume today is 24 September 2026.

Question 1

Question 1. Given a MongoDB collection 'orders' with documents containing fields: `userId`, `items` (array of `{product, quantity, price}`), and `orderDate`, use the `$match` stage to find all orders placed in the last 30 days.

Sample data (used in Questions 1 and 2):

```
use shop_db
```

```
db.orders.insertMany([
  { _id: 1, userId: 1, orderDate: ISODate("2026-09-20"),
    items: [ { product: "Laptop Bag", quantity: 1, price: 999 },
              { product: "Mouse",      quantity: 2, price: 499 } ] },
  { _id: 2, userId: 2, orderDate: ISODate("2026-09-10"),
    items: [ { product: "Headphones", quantity: 1, price: 1999 } ] },
  { _id: 3, userId: 1, orderDate: ISODate("2026-09-01"),
    items: [ { product: "Keyboard",   quantity: 1, price: 1299 } ] },
  { _id: 4, userId: 3, orderDate: ISODate("2026-07-15"),
    items: [ { product: "Phone Case", quantity: 3, price: 299 } ] },
  { _id: 5, userId: 2, orderDate: ISODate("2026-08-30"),
    items: [ { product: "Charger",    quantity: 2, price: 599 } ] }
])
```

Answer: calculate the date 30 days ago, then use `$match` with `$gte` to keep only orders on or after that date.

```
const thirtyDaysAgo = new Date(Date.now() - 30 * 24 * 60 * 60 * 1000)

db.orders.aggregate([
  { $match: { orderDate: { $gte: thirtyDaysAgo } } }
])
```

`Date.now()` gives the current time in milliseconds, and `30 * 24 * 60 * 60 * 1000` is 30 days in milliseconds. With today as 24 September 2026, the cut-off is 25 August 2026.

Output (order 4 from 15 July is left out because it is older than 30 days):

```
[
  {
    _id: 1,
    userId: 1,
    orderDate: ISODate('2026-09-20T00:00:00.000Z'),
    items: [
      { product: 'Laptop Bag', quantity: 1, price: 999 },
      { product: 'Mouse',     quantity: 2, price: 499 }
    ]
  },
  {
    _id: 2,
    userId: 2,
    orderDate: ISODate('2026-09-10T00:00:00.000Z'),
    items: [ { product: 'Headphones', quantity: 1, price: 1999 } ]
  },
  {
    _id: 3,
    userId: 1,
    orderDate: ISODate('2026-09-01T00:00:00.000Z'),
    items: [ { product: 'Keyboard',   quantity: 1, price: 1299 } ]
  },
  {
    _id: 4,
    userId: 3,
    orderDate: ISODate('2026-07-15T00:00:00.000Z'),
    items: [ { product: 'Phone Case', quantity: 3, price: 299 } ]
  },
  {
    _id: 5,
    userId: 2,
    orderDate: ISODate('2026-08-30T00:00:00.000Z'),
    items: [ { product: 'Charger',    quantity: 2, price: 599 } ]
  }
]
```

```

    _id: 5,
    userId: 2,
    orderDate: ISODate('2026-08-30T00:00:00.000Z'),
    items: [ { product: 'Charger', quantity: 2, price: 599 } ]
  }
]

```

Points to remember:

- orderDate must be stored as a Date. If it is stored as a string, the comparison will not work correctly.
- The results depend on the day you run the query, because "last 30 days" moves forward every day.
- On MongoDB 5.0 and above you can also do this inside the pipeline: { \$match: { \$expr: { \$gte: ["\$orderDate", { \$dateSubtract: { startDate: "\$\$NOW", unit: "day", amount: 30 } }] } } }.

Question 2

Question 2. Using the 'orders' collection, write an aggregation pipeline that groups orders by userId and calculates the total amount spent by each user using the \$group stage. Hint: Use \$sum to total the value of all items in each user's orders.

Answer: each order has an items array, so first use \$unwind to turn each item into its own document. Then \$group by userId and \$sum the value of each item (quantity multiplied by price).

```

db.orders.aggregate([
  { $unwind: "$items" },
  { $group: {
    _id: "$userId",
    totalSpent: { $sum: { $multiply: ["$items.quantity", "$items.price"] } }
  } },
  { $sort: { totalSpent: -1 } }
])

```

Output:

```

[
  { _id: 1, totalSpent: 3296 },
  { _id: 2, totalSpent: 3197 },
  { _id: 3, totalSpent: 897 }
]

```

How the totals are worked out:

userId	Calculation	Total
1	(1 x 999) + (2 x 499) + (1 x 1299)	3296
2	(1 x 1999) + (2 x 599)	3197
3	(3 x 299)	897

Points to remember:

- In \$group, the _id field is the grouping key, so here _id holds the userId.
- \$sort is optional. It only orders the result from the highest spender to the lowest.
- This totals all orders. To total only the last 30 days, put the \$match stage from Question 1 at the start of the pipeline.
- Alternative without \$unwind: { \$group: { _id: "\$userId", totalSpent: { \$sum: { \$sum: { \$map: { input: "\$items", as: "i", in: { \$multiply: ["\$\$i.quantity", "\$\$i.price"] } } } } } } } }.

Question 3

Question 3. Create an aggregation pipeline on a 'restaurants' collection (fields: name, cuisine, city, rating) to find the top 3 highest-rated cafes in Ahmedabad. Use \$match for cuisine, \$sort for rating, and \$limit for top results.

Sample data:

```
db.restaurants.insertMany([
  { name: "The Coffee Corner", cuisine: "Cafe", city: "Ahmedabad", rating: 4.8 },
  { name: "Cafe Mocha House", cuisine: "Cafe", city: "Ahmedabad", rating: 4.6 },
  { name: "Brew and Bites", cuisine: "Cafe", city: "Ahmedabad", rating: 4.4 },
  { name: "Latte Lane", cuisine: "Cafe", city: "Ahmedabad", rating: 4.1 },
  { name: "Beans Central", cuisine: "Cafe", city: "Mumbai", rating: 4.9 },
  { name: "Spice Route", cuisine: "North Indian", city: "Ahmedabad", rating: 4.7 }
])
```

Answer: \$match keeps only cafes in Ahmedabad, \$sort with -1 puts the highest rating first, and \$limit keeps the first 3.

```
db.restaurants.aggregate([
  { $match: { cuisine: "Cafe", city: "Ahmedabad" } },
  { $sort: { rating: -1 } },
  { $limit: 3 },
  { $project: { _id: 0, name: 1, city: 1, rating: 1 } }
])
```

The \$project stage at the end is optional. It only hides the _id field to keep the output short.

Output:

```
[
  { name: 'The Coffee Corner', city: 'Ahmedabad', rating: 4.8 },
  { name: 'Cafe Mocha House', city: 'Ahmedabad', rating: 4.6 },
  { name: 'Brew and Bites', city: 'Ahmedabad', rating: 4.4 }
]
```

What was left out: Beans Central (a cafe, but in Mumbai), Spice Route (in Ahmedabad, but not a cafe) and Latte Lane (a cafe in Ahmedabad, but only 4th by rating).

Why the order matters: \$match should come first so later stages work on fewer documents, and \$sort must come before \$limit so that the 3 kept are the highest rated.

Question 4

Question 4. In a 'movies' collection (fields: title, genre, boxOffice), use \$group to calculate the total box office collection for each genre and then use \$project to display only genre and totalCollection fields in the output. Constraint: Do not include the default _id field in your final projection.

Sample data (the box office numbers are made-up sample values for practice, not real figures):

```
db.movies.insertMany([
  { title: "Star Voyage", genre: "Sci-Fi", boxOffice: 500 },
  { title: "Space Frontier", genre: "Sci-Fi", boxOffice: 700 },
  { title: "Laughter Riot", genre: "Comedy", boxOffice: 300 },
  { title: "Comedy Nights", genre: "Comedy", boxOffice: 250 },
  { title: "Midnight Chase", genre: "Action", boxOffice: 900 },
  { title: "Iron Fist Returns", genre: "Action", boxOffice: 650 }
])
```

Answer:

```
db.movies.aggregate([
  { $group: {
    _id: "$genre",
    totalCollection: { $sum: "$boxOffice" }
  } },
  { $sort: { totalCollection: -1 } },
  { $project: {
```

```

        _id: 0,
        genre: "$_id",
        totalCollection: 1
    } }
  })

```

Output:

```

[
  { totalCollection: 1550, genre: 'Action' },
  { totalCollection: 1200, genre: 'Sci-Fi' },
  { totalCollection: 550, genre: 'Comedy' }
]

```

How it works:

- \$group makes one output document per genre. The genre becomes the _id, and \$sum adds up the boxOffice values.
- After \$group, the genre is stored in _id, so \$project copies it into a new field with genre: "\$_id".
- _id: 0 removes the default _id field, which satisfies the constraint. MongoDB allows _id: 0 together with other included fields.
- The \$sort stage is optional. It is added so the output order is predictable, because \$group does not guarantee any order.
- The order of fields inside the printed documents can differ slightly in your shell. Only the field names and values matter.

Question 5

Question 5. Use ChatGPT or GitHub Copilot to generate an aggregation pipeline that produces a daily sales report from a 'payments' collection (fields: userId, amount, paymentDate), showing total sales per day for the last week. Paste the generated code and briefly describe any changes you made to fit your collection's structure.

I cannot use ChatGPT or Copilot for you, and your teacher will expect your own screenshot and pasted code. Do it yourself like this:

Step 1: Paste this prompt into ChatGPT or Copilot:

"Write a MongoDB aggregation pipeline for a payments collection with fields userId, amount and paymentDate that shows total sales per day for the last 7 days."

Step 2: Take a screenshot of the prompt and the response, and paste the generated code into your submission. The tool may give slightly different code from the typical example below.

Step 3: Typical generated code (uses generic field names, so it needs adapting):

```

db.payments.aggregate([
  { $match: { createdAt: { $gte: new Date(new Date().setDate(new Date().getDate() - 7)) } } },
  { $group: {
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$createdAt" } },
    totalSales: { $sum: "$total" }
  } },
  { $sort: { _id: 1 } }
])

```

Step 4: Adapted code for the payments collection (submit this along with the generated code):

```

const weekAgo = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000)

db.payments.aggregate([
  { $match: { paymentDate: { $gte: weekAgo } } },
  { $group: {
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$paymentDate", timezone: "Asia/Kolkata" } },
    totalSales: { $sum: "$amount" }
  } }
])

```

```

        transactions: { $sum: 1 }
    } },
    { $sort: { _id: 1 } },
    { $project: { _id: 0, date: "$_id", totalSales: 1, transactions: 1 } }
  ])

```

Sample data to test with (the payment on 10 September is older than a week and should be left out):

```

db.payments.insertMany([
  { userId: 1, amount: 500, paymentDate: ISODate("2026-09-24T09:15:00Z") },
  { userId: 2, amount: 1200, paymentDate: ISODate("2026-09-24T14:40:00Z") },
  { userId: 3, amount: 300, paymentDate: ISODate("2026-09-23T11:00:00Z") },
  { userId: 1, amount: 800, paymentDate: ISODate("2026-09-22T16:20:00Z") },
  { userId: 2, amount: 450, paymentDate: ISODate("2026-09-22T18:05:00Z") },
  { userId: 4, amount: 2000, paymentDate: ISODate("2026-09-20T10:30:00Z") },
  { userId: 3, amount: 700, paymentDate: ISODate("2026-09-18T08:45:00Z") },
  { userId: 1, amount: 650, paymentDate: ISODate("2026-09-10T12:00:00Z") }
])

```

Output:

```

[
  { totalSales: 700, transactions: 1, date: '2026-09-18' },
  { totalSales: 2000, transactions: 1, date: '2026-09-20' },
  { totalSales: 1250, transactions: 2, date: '2026-09-22' },
  { totalSales: 300, transactions: 1, date: '2026-09-23' },
  { totalSales: 1700, transactions: 2, date: '2026-09-24' }
]

```

Changes I made to fit the collection:

- Renamed the fields to match the collection: createdAt became paymentDate and total became amount.
- Added timezone: "Asia/Kolkata" to \$dateToString so days are counted in Indian time. Without it, MongoDB uses UTC, and late-evening payments could land on the wrong day.
- Replaced the setDate() date arithmetic with a simpler Date.now() calculation stored in a weekAgo variable.
- Added a transactions count using { \$sum: 1 } so the report shows how many payments made up each day's total.
- Added a final \$project stage to rename _id to date and give a cleaner report.

Points to remember: paymentDate must be stored as a Date. Days with no payments (19 and 21 September here) do not appear, because \$group only creates rows for days that have data. Newer MongoDB versions offer \$densify to fill in the missing days with zero.

Stage Summary

Stage	What it does
\$match	Filters documents, like a WHERE clause
\$unwind	Splits an array so each element becomes its own document
\$group	Groups documents by a key and calculates totals (sum, count, and so on)
\$sort	Sorts documents (1 for ascending, -1 for descending)
\$limit	Keeps only the first n documents
\$project	Chooses, renames or calculates the fields in the output